Essae Weighing Scale & Label Printer - User Space Driver Documentation (v2.0)

Prepared By	Venkatesh M	venkatesh.muninagaraju@essae.com
Reviewed By	Nagarajan V	nagarajan@essae.com

1. System Overview

This document describes the architecture, command interface, communication protocol, and usage of the user-space Linux driver for the Essae weighing scale and label printer, designed for RK3568-based systems.

The system supports two primary functionalities:

- Reading weight from a serially connected Essae weighing scale
- Printing labels using an Essae label printer by parsing. LFT label template files and combining them with data from a JSON file containing 1–96 data fields

In v2.0, the system enhancements include:

- . LFT label templates are stored and retrieved from a local SQLite database (**SQL_LFT_Files.db**)
- The GUI now passes the full JSON data (IDs 1–96) file along with the selected. LFT slot and barcode number.

The system operates in two modes:

- **Server Mode:** TCP server on port 8888 to handle remote commands from the GUI or other tools
- The server is also configured to run in the background as a systemd service, always listening on port 8888. Users only need to run the Python GUI to interact with it.

2. Ports & Device Paths

Component	Port Path	Baud Rate	Description
Label Printer	/dev/ttyS0	115200	Receives formatted print commands
Weighing Scale	/dev/ttyS4	9600	Sends/receives ASCII or binary weight commands
Server	TCP Port 8888	-	Accepts remote printer/scale requests from clients

3. Communication Protocol Server Mode

The server listens on TCP port 8888. Based on the command received, it performs either of the following operations:

Printer Mode:

MODE:PRINTER

<JSON file path> → **Example: Select the JSON file containing data IDs 1–96**

<LFT slot number> → **Example: Select the label design slot (1–99) from SQL_LFT_Files.db**

<Barcode ID (1–96)> → **Example: Select which JSON field to use for the ~B barcode command**

- In **v2.0**, the server expects the full JSON file containing all 1–96 data fields.
- It loads the selected. LFT label design from the database and replaces the placeholders (~T, ~V, ~B) using the provided JSON data.
- The barcode number specifies which JSON key will be used for barcode generation (~B command only).

Scale Mode:

MODE:WEIGHT

<command> → **Example: RD_WEIGHT, XC_TARE, XC_REZERO, etc.**

Each command is followed by response(s) sent over the same socket to the client.

4. Weighing Scale Commands

The following commands are parsed and sent over **/dev/ttyS4:**

Command	Description	ASCII/Hex Value
RD_WEIGHT	Reads current weight	0x05
XC_TARE	Sends 'T' command (Tare)	'T' / 't'
XC_REZERO	Sends 'Z' command (Zero scale)	0x10
XC_SON	Enter calibration mode	0x12
XC_KEYCALxxxxx	Set calibration weight in grams	0x13
XC_CALZERO	Calibration - Zero	0x14
XC_CALSPAN	Calibration - Span	0x15
XC_CALIBRATE	Finalize calibration	0x16
XC_RDRAWCT	Read raw count	0x11
XC_RESTART	Restarts the scale	0x1C
RD_TECHSPEC	Read technical parameters	0x19
WR_TECHSPEC	Write technical parameters	0x18
RD_CUSSPEC	Read custom configuration	0x1B
WR_CUSSPEC	Write custom configuration	0x1A

Each command is sent as ASCII bytes. Delays and responses are handled with timeouts and retries.

5. Label Printer Commands

The printer receives formatted .LFT files containing drawing/printing instructions Refer the **Label Report Description language R2 PDF**. Each command starts with a tilde ~.

Command	Description
~S	Label size (~S,width_mm,height_mm)
~A	Define clear area
~T	Print fixed text
~V	Print variable (from JSON) text
~B	Print barcode data (from JSON)
~R	Draw rectangle
~C	Draw circle
~c	Escape Codes
~d	Print bitmap image
~I	Set print intensity (100-140)
~Y	Delay printing for specified ms
~P	Print page (finalize buffer)
~e	Read response from printer
~s	Line Spacing

6. Command Format Details Refer Label Report Description language R2.PDF

~T - Fixed Text

Command, x location [mm], y location [mm], angle [0: 90:180:270], font [1:2], x magnify [1-6], y magnify [1-6], text, data length, offset, justify [N: L: C: R], lines, line spacing [mm], mode [C: F: W: X: I: U], Pint Status [0-5]

~T,11.875,0.125,0,2,1,2,Heritage Fresh,14,0,N,1,2.500,E,1

~V - Variable Text

Command, x location [mm], y location [mm], angle [0: 90: 180: 270], font [1: 2], x magnify [1-6], y magnify [1-6], data ID, data, data length, offset, justify [N: L: C: R], lines, line spacing [mm], mode [C: F: W: X: I: U], Pint Status [0-5]

~V,0,5.25,0,2,1,1,2,Onion Medium,36,0,C,1,2.500,W,1

~B - Barcode from JSON

Command, x location [mm], y location [mm], angle [0, 90, 180, 270], HRI font [1, 2], barcode width [0.125, 0.250, 0.375, 0.500, 0.625], barcode height [mm], data,

data length, offset, justify [N: L: C: R], barcode type [EAN13, CODE128, CODE128A, CODE128B, CODE128C: QRCODE], HRI position [N: T: B: 2],

mode [C: F: W: X: I], Pint Status [0-5],

~B,1.25,7.5,0,2,0.25,8,12345678901234567890123456789012345678,28,0,N,CODE128,B,W,1,

7. Data Flow

Printing (v2.0):

Client GUI → TCP → Server
→ Parse full JSON (IDs 1–96), selected LFT slot, and Barcode ID
→ Load LFT from SQL_LFT_Files.db
→ Open /dev/ttyS0
→ Send ESC/POS Commands
→ Label Printer

Weighing:

Client GUI → TCP → Server
→ Send weighing command over /dev/ttyS4
→ Get scale response
→ Return to client

8. GUI Python Client Integration

The **Essae_WSLPR_client_v2.0.py** GUI connects to the server via TCP on port 8888 and provides the following functionality:

Label Printer LFT Slot Management

- **Add LFT:** Upload .LFT file to SQLite DB (SQL_LFT_Files.db)
- **Edit LFT:** Modify .LFT content inside the GUI
- **Delete LFT:** Remove an LFT slot from the database
- **Rename LFT:** Rename LFT slot name in database *(optional feature)*

Barcode Management (v2.0)

- **Add Barcode:** Save a new barcode name and ID into the barcodes table in the same database
- **Edit Barcode:** Modify existing barcode name or associated ID
- **Delete Barcode:** Remove a barcode entry from the table
- **Barcode DB Table:** Stored in SQL_LFT_Files.db in a separate table (barcode_templates)

Barcode and Print Workflow

- **Barcode selection:** Choose a data ID (1–96) from JSON to use for the ~B command
- **Print function:** Sends selected. LFT slot, full JSON data, and barcode ID to the server over TCP

Weighing Scale Controls

- Read weight
- Tare / Zero / Restart
- Calibration (multi-step process)
- Read/write technical and custom specifications

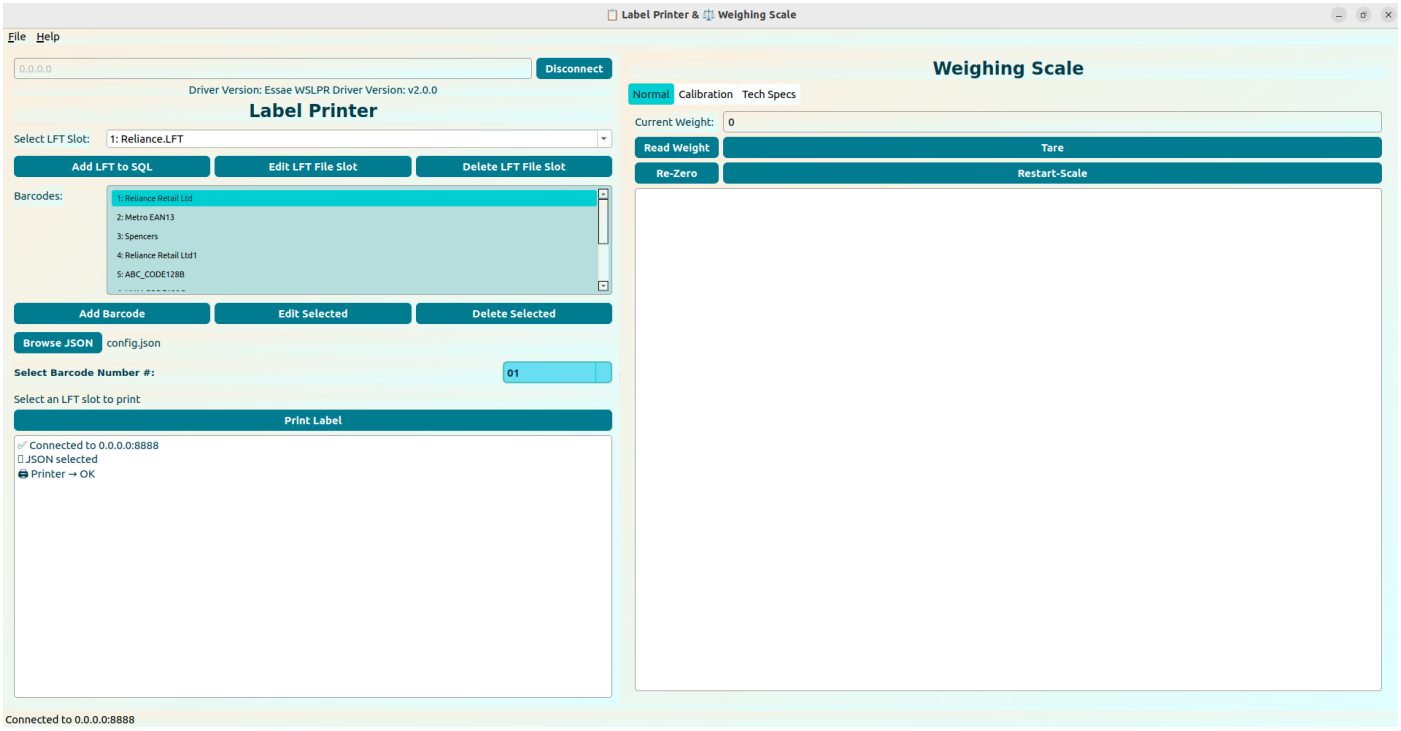
GUI Features

- User-friendly controls and live action logs
- Response logs for Normal, Calibration, and Tech tabs
- Displays connection status
- Disables print if JSON is not selected or if not connected
- Uses SQL_LFT_Files.db with the following schemas:

Table: **lft_files**(slot INTEGER PRIMARY KEY, name TEXT, content BLOB)
Table: **barcodes_templates**(id INTEGER PRIMARY KEY, name TEXT, data TEXT)

To run the GUI:

$ python3 Essae_WSLPR_client_v2.0.py



9. Required Packages & Dependencies

Install the following packages on **Ubuntu 22.04**:

For Server:

$ sudo apt install build-essential libjson-c-dev libsqlite3-dev

For Client GUI:

$ sudo apt install python3 python3-pyqt5 sqlitebrowser
$ pip3 install PyQt5

✅ *Note (v2.0): The SQLite database stores both .LFT label templates and barcode metadata used for printing.*

10. Server and Service Modes

Server Mode:

To run the label printing server manually:

$ gcc Essae_WSLPR_server_v2.0.c -o Essae_WSLPR_server_v2.0 -ljson-c -lsqlite3 -lm
$ ./Essae_WSLPR_server_v2.0

Service Mode:

To run the server automatically in the background on every system boot:

1. **Create the service file:**

$ sudo nano /etc/systemd/system/essae_server.service

2. **Paste the following:**

```
[Unit]
Description=Essae WSLPR TCP Server v2.0
After=network.target

[Service]
ExecStart=/home/essae/Documents/Essae_Data/Projects/Essae_Rockchip_RK3568_Seavo/Essae_WSLPR_Driver_Code/Essae_WSLPR_server_v2.0
Restart=always
User=essae
WorkingDirectory=/home/essae/Documents/Essae_Data/Projects/Essae_Rockchip_RK3568_Seavo/Essae_WSLPR_Driver_Code

[Install]
WantedBy=multi-user.target
```

3. **Reload and start:**

```
$ sudo systemctl daemon-reload
$ sudo systemctl enable essae_server.service
$ sudo systemctl start essae_server.service
```

4. **Check server status:**

```
$ journalctl -u essae_server.service --no-pager --since "5 minutes ago"
```

✅ Ensure the SQLite .db path is valid. Use an absolute path or keep the database in the server's working directory.

To run the GUI client:

```
$ python3 Essae_WSLPR_client_v2.0.py
```

11. Error Handling

- Missing serial ports are logged as **warnings**, allowing the system to continue
- Invalid .LFT commands are **skipped gracefully**
- JSON is decoded using libjson-c; **missing keys** fall back to defaults
- GUI **disables print** if JSON is not selected or TCP is disconnected
- All operations log responses to assist debugging and traceability

12. Appendix – Database Structure (v2.0)

Database File: SQL_LFT_Files.db

Label Template Table:

Table: lft_files
Columns:
- slot INTEGER PRIMARY KEY
- name TEXT
- content BLOB

Used to save. LFT label design templates uploaded via GUI.

Barcode Table (v2.0):

Table: barcodes_templates
Columns:
- id INTEGER PRIMARY KEY
- name TEXT
- data TEXT

Used to manage user-defined barcodes stored in the GUI — selectable at print time.

You can access the database using:

$ sqlite3 SQL_LFT_Files.db

or use a visual tool like sqlitebrowser.